AICCE: AI-Driven Compliance Checker Engine

Mohammad Wali Ur Rahman, Martin Manuel Lopez, Lamia Tasnim Mim, Carter Farthing, Julius Battle, Kathryn Buckley, and Salim Hariri, *Senior Member, IEEE*

***Abstract*—For digital infrastructure to be safe, compatible, and standards-aligned, automated communication protocol compliance verification is crucial. Nevertheless, current rule-based systems are becoming less and less effective since they are unable to identify subtle or intricate noncompliance, which attackers frequently use to establish covert communication channels in IPv6 traffic. To automate IPv6 compliance verification, this article presents the artificial intelligence-driven compliance checker engine (AICCE), a novel generative system that combines dual-architecture reasoning and retrieval-augmented generation (RAG). Specification segments pertinent to each query can be efficiently retrieved thanks to the semantic encoding of protocol standards into a high-dimensional vector space. Based on this framework, AICCE offers two complementary pipelines: 1) explainability mode, which uses parallel LLM agents to render decisions and settle disputes through organized discussions to improve interpretability and robustness; and 2) script execution mode, which converts clauses into Python rules that can be executed quickly for dataset-wide verification. With the debate mechanism enhancing decision reliability in complicated scenarios and the script-based pipeline lowering per-sample latency, AICCE achieves accuracy and F1-scores of up to 99% when tested on IPv6 packet samples across 16 cutting-edge generative models. By offering a scalable, auditable, and generalizable mechanism for identifying both routine and covert noncompliance in dynamic communication environments, our results show that AICCE overcomes limitations of conventional rule-based compliance checking systems.**

***Impact Statement*—Traditional methods of compliance verification have become inadequate due to the growing complexity of communication protocol standards such as IPv6, particularly in high-assurance contexts where accuracy is of utmost importance. The methodology presented in this work combines semantic retrieval and generative multiagent reasoning to automate the standards compliance verification process. The system provides scalable, interpretable, and model-agnostic compliance evaluations by integrating protocol requirements into a vectorized knowledge base and using retrieval-augmented generation. Decision robustness is further increased by incorporating a multiagent discussion system, which resolves unclear or contradictory conclusions. A thorough empirical analysis using several cutting-edge generative models shows that this method consistently and accurately verifies IPv6 packet conformance. The framework's ability to semantically align real-world data with formal protocol standards represents a significant step toward dependable, AI-driven auditing tools. It is applicable not only to IPv6 but also to other domains that need dynamic, context-aware standard conformance analysis, such as defense communications, cybersecurity, and networking systems regulatory compliance.**

***Index Terms*—Deep learning, generative AI, IPv6, multiagent system (MAS), protocol compliance verification, query resolution, retrieval augmented generation (RAG), vector database.**

I. Introduction

THANKS to the internet's exponential growth, the proliferation of connected devices, and new application paradigms, Internet Protocol version 6 (IPv6) has become an essential part of contemporary communication systems. Central to IPv6's adoption is its vastly expanded address space, which alleviates the exhaustion issues faced by IPv4 while enabling more flexible topological assignments [1], [2]. In addition to the expanded address space, IPv6 offers more efficient routing and opens the door for new network-layer services with its strong extension methods and simplified header structures [1], [3]. Large-scale service providers, mobile networks, and cutting-edge technologies such as the Internet of Things (IoT) now depend on IPv6.

Despite its obvious benefits, maintaining IPv6's proper and secure operation in a variety of real-world settings is still very difficult. A complex web of interconnected requests for comments (RFCs) that specify different behaviors, extensions, and updates must be navigated by implementers and network operators [1], [2], [3]. The complex specifications that must be standardized across heterogeneous systems frequently span hundreds of pages due to the subtle dependencies and complex requirements. Performance problems, security flaws, or incompatibilities can result from minor errors in the interpretation of important protocol clauses [4], [5], [6]. Because IPv6 standards are dynamic and complex, and because new RFCs and

Received 18 October 2025; revised 20 February 2026; accepted 1 April 2026. Date of publication 10 April 2026; date of current version 31 August 2026. This work was supported in part by the National Science Foundation (NSF) under Grant 1624668, Grant 1921485, Grant 1303362 (Scholarship-for-Service), Grant 2413009 and WISPER Center; and in part by the 2025 Technology and Research Initiative Fund/National Security Systems Initiative administered by the University of Arizona, Office of Research and Partnerships under Proposition 301, the Arizona Sales Tax for Education Act, in 2000. This article was recommended for publication by Associate Editor Qinghua Lu upon evaluation of the reviewers' comments. *(Corresponding author: Mohammad Wali Ur Rahman.)*

Mohammad Wali Ur Rahman, Martin Manuel Lopez, and Salim Hariri are with the Department of Electrical and Computer Engineering, University of Arizona, Tucson, AZ 85721 USA (e-mail: mwrahman@arizona.edu; martinmlopez@arizona.edu; hariri@arizona.edu).

Lamia Tasnim Mim is with Avirtek, Inc., Tucson, AZ 85712 USA (e-mail: lamiya.tasnim@avirtek.com).

Carter Farthing, Julius Battle, and Kathryn Buckley are with the Joint Interoperability Test Command (JITC), United States Department of Defense, Sierra Vista, AZ 85613 USA (e-mail: carter.j.farthing.civ@mail.mil; julius.a.battle.civ@mail.mil; kathryn.v.buckley.ctr@mail.mil).

This article has supplementary downloadable material available at https://doi.org/10.1109/TAI.2026.3682692, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3682692

2691-4581 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

clarifications are frequently released, they surpass the capabilities of conventional compliance verification techniques such as static rule-checking tools or manual reviews by subject-matter experts [7], [8], [9], [10], [11].

Large language models (LLMs) have recently demonstrated impressive abilities to comprehend and produce human language, opening the door to sophisticated applications in code generation, dialogue systems, and question answering [12], [13], [14]. When combined with retrieval-augmented generation (RAG) techniques, these models can dynamically access external knowledge bases during inference, making their outputs more accurate and contextually grounded [15]. This method is particularly well-suited for protocol compliance verification, where the relevant sections of formal documents may be scattered across multiple RFCs. By embedding these documents into a high-dimensional vector space [16], an LLM can retrieve highly specific clauses, parse them in context, and then generate a compliance verdict. However, applying RAG-based methods to IPv6 presents several nontrivial obstacles. The language used in protocol documents is both technical and subtle, rife with cross-references and specialized terminology. Furthermore, some clauses require careful interpretation of ambiguous text because they overlap or impose requirements that are partially contradictory.

We propose AICCE, an improved standard compliance verification framework that combines a structured debate mechanism with multiagent generative reasoning to overcome these limitations. Rather than depending on a single model to determine compliance, our system coordinates the actions of several generative agents to work simultaneously. Each agent either makes an independent compliance decision or extracts the important rules for compliance verification based on the context of the retrieved IPv6 specification section. A debate phase is initiated in situations where there is disagreement or uncertainty [17], [18], which enables these agents to reconsider the relevant evidence. This iterative process builds a stronger consensus as agents review potential oversights and improve their decisions. In addition to enhancing accuracy in borderline situations, this method provides an operationally transparent view of the decision-making process by identifying the exact RFC clauses and semantic interpretations that inform each decision.

We empirically validate our framework on a dataset of 1500 IPv6 packet samples, carefully curated to reflect real-world traffic diversity. We benchmark 16 state-of-the-art generative models, assessing their performance on compliance classification metrics such as accuracy and F1-score. Our results show that the highest-performing models achieve scores above 0.99, with the debate mechanism reducing errors in challenging or ambiguous cases. Moreover, a qualitative review of the multiagent interactions underscores how this debate mechanism uncovers hidden assumptions and offers deeper explanations for why an edge case is deemed compliant or noncompliant [14], [17].

In summary, the primary contributions of this article are.

1) A retrieval-augmented generative AI framework specifically tailored to IPv6 compliance verification, based on a vectorized repository of official standard documents.
2) A multiagent debate mechanism that systematically resolves uncertain or conflicting judgments, thereby improving both classification accuracy and interpretability.
3) A comprehensive experimental comparison of 16 generative models, highlighting the efficacy of structured debate in high-stakes compliance contexts and offering insights into model behaviors.

While IPv6 serves as a critical focal point for our investigations, the proposed methodology is readily adaptable to other complex technical standards and regulatory environments. Accordingly, this work lays a foundation for broader applications in cybersecurity, regulatory compliance, and secure communications infrastructure. The remainder of the article is organized as follows. Section II provides a review of related research on network protocol verification and multiagent reasoning. Section III presents the system architecture and core algorithms that drive our framework. Section IV details the experimental setup, including data collection procedures and evaluation metrics. In Section V, we analyze and interpret the empirical findings. Finally, Section VI concludes the article and outlines directions for future work.

II. Related Articles

In this section, we situate our work in the broader landscape of network protocol validation and multiagent reasoning. We review existing methods, highlight their shortcomings in addressing the complexities of IPv6 compliance, and show how our proposed approach bridges crucial gaps.

A. Network Protocol Validation

Network protocols underpin virtually all internet communications, and verifying their correct implementation has been a focus of research for decades. Classical validation techniques rely on either extensive human oversight or purely automated testing, each with its own drawbacks. One critical area where accurate network validation is indispensable is in detecting covert communication channels within network traffic where hidden information is embedded within otherwise legitimate traffic [5], [25]. Pattern and signature-based engines such as SNORT and Suricata [7], [8] are effective for known misconfigurations but frequently miss semantic deviations characteristic of covert transmissions [4], [6].

Historically, protocol validation has often depended on domain experts meticulously comparing protocol behaviors against official specifications. Early network implementations for TCP/IP [26] and similar standards were checked through labor-intensive code reviews and conformance matrices, which spelled out allowed packet types, header fields, and state transitions. Over time, rule-based engines emerged to codify basic checks: for instance, verifying that IPv6 extension headers follow a specified ordering or confirming that header field values adhere to defined limits. While scalable, such static checks struggle with ambiguous or interdependent clauses spread across evolving RFCs [2], [3]. Formal verification tools such as SPIN [9] or Tamarin [10] bring rigor through state-space exploration, but they require substantial modeling expertise

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

and are costly to scale to evolving protocol implementations. Complementary to exhaustive model checking, statistical model checking (SMC) estimates whether a property holds by running many randomized simulations and providing results with statistical confidence [19], [20], often scaling better when full state-space exploration is infeasible. Toolchains and approaches such as statistical checking of real-time/probabilistic systems and probabilistic verification frameworks (e.g., PRISM) have been widely used to analyze quantitative or stochastic properties [21], [22]. These approaches reduce up-front modeling brittleness compared with fully exhaustive exploration, but still require formalization of the system model and properties, which can be costly for evolving RFC ecosystems. Fuzzing [11] via tools such as Boofuzz [23] and AFL [24] exposes robustness defects, yet does not quantify comprehensive standards compliance.

IPv6 is a particularly challenging use case for any verification method. RFC 2460 [3] was the first to outline its definition; RFC 8200 [1] and IPv6 Addressing Architecture documents [2] were among the many RFCs that revised and expanded it. These articles frequently relate to one another and contain complex rules for neighbor identification, path MTU discovery, address scopes, extension header chaining, and more. As a result, it is easy for implementers to miss subtle interactions between several standard parts. Until complicated, real-world circumstances, such as tunneling or traffic in heavily meshed topologies, disclose hidden assumptions, errors or ignored needs may go undetected [27]. Although manual validation, mathematically grounded formal verification, and fuzz testing each address specific facets, they fall short of comprehensive, end-to-end assurance of clause-faithful IPv6 compliance at scale. Manual reviews do not scale, formal methods are costly and brittle to evolving implementations, and fuzzing reveals robustness defects rather than systematic conformance to interdependent RFC clauses. Moreover, none of these approaches reliably captures the semantic reasoning required to interpret cross-referenced, context-dependent requirements in the IPv6 standards.

B. Multiagent Reasoning

The field of multiagent AI has grown rapidly in tandem with improvements in protocol validation. Modern applications take advantage of the combined intelligence of several independent agents, each of which offers unique insights or partial answers to complex problems.

In multiagent AI systems, multiple intelligent agents collaborate (or sometimes compete) to accomplish tasks whose complexity may surpass the capability of a single agent. Each agent acts on local observations yet may also exchange information or learned parameters to enhance collective outcomes [28]. This paradigm has proven effective in highly distributed settings such as federated learning [29], where data is scattered across different nodes or devices and must be processed without compromising privacy. Multiagent reinforcement learning (RL-MAS) extends single-agent RL principles to shared environments with multiple decision-makers [30]. Agents iteratively refine their policies via trial and error, adapting to evolving conditions and the actions of other agents.

Swarm intelligence approaches, such as particle swarm optimization [31] and ant colony optimization [32], exemplify how decentralized groups can coordinate to solve optimization problems by exploring a wide range of solutions in parallel. Negotiation-based models, meanwhile, allocate resources and resolve conflicts through formal agreement mechanisms, often informed by game-theoretic insights [33], [34]. Among these, actor-critic methods [35] have gained attention because they couple policy-based and value-based learning in a single framework, allowing for dynamic adjustments based on ongoing evaluative feedback. This iterative refinement process is especially valuable in scenarios that demand adaptive, high-precision decision-making.

While the literature on multiagent systems (MAS) is extensive, applications to network protocol compliance remain limited. Typical MAS formulations presume well-defined environments and explicit payoffs, whereas IPv6 conformance requires interpreting interlinked, sometimes ambiguous RFC clauses. Domain-agnostic or heuristic debate structures thus risk shallow, ungrounded arguments unless anchored in standards text; without robust retrieval, agent discussions can miss critical evidence and yield spurious conclusions.

To address this gap, prior advances in retrieval-augmented generation (RAG) [15] and structured debate [17] motivate clause-grounded, multiagent reasoning for compliance. Embedding IPv6 RFCs enables agents to cite explicit passages and contest interpretations (e.g., extension-header sequencing), promoting consensus that is both auditable and accurate. In contrast, black-box single-agent methods obscure evidentiary chains, undermining correctness and interpretability in high-stakes networking. In sum, whereas manual checks, fuzzing, and formal proofs miss cross-document semantics, and generic MAS lacks domain grounding, RAG-backed debate directly targets clause-faithful verification; subsequent sections detail the architecture and empirical benchmarks. Table I compares representative protocol-validation approaches for IPv6. *Automated* denotes whether the pipeline runs end-to-end at runtime once its required artifacts (e.g., rules, formal models, or specification encodings) are in place, without interactive human involvement during checking. *Minimal expert setup* captures the up-front and maintenance effort to create and update those artifacts as RFCs evolve; thus, model checking is automated at runtime but typically has high expert setup cost.

III. Methodology

This section presents the complete methodology behind our generative AI-based IPv6 protocol compliance framework, which is implemented in two distinct architectures: the *explainability mode* for deep interpretability and thorough clause-level compliance verification, and the *script execution mode* for low-latency execution. Both pipelines rely on semantic document embeddings, vector-based retrieval, and dynamic standard interpretation. Their complementary strengths, where one favors depth and auditability, the other speed and automation, enable

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QUALITATIVE COMPARISON OF NETWORK PROTOCOL VALIDATION METHODS ACROSS KEY CRITERIA

System/Method	Detects Covert Channels	Handles RFC Ambiguities	Scales to IPv6 Extensions	Traceable / Explainable	Automated	Cross-RFC Reasoning	Minimal Expert Setup	Semantic Checking
Manual Checks (Expert Review)	✓	✓	✗	✓	✗	✓	✗	✓
Rule-Based Engines [7], [8]	✗	✗	✗	✗	✓	✗	✓	✗
Formal Verification (Model Checking) [9], [10], [19], [20], [21], [22]	✓	✓	✗	✓	✓	✗	✗	✗
Fuzz Testing [11], [23], [24]	✓	✗	✓	✗	✓	✗	✓	✗
Single-Agent System [15]	✗	✓	✓	✗	✓	✗	✓	✓
Ours: AICCE	✓	✓	✓	✓	✓	✓	✓	✓

flexible deployment across different operational constraints. The overall system architecture is illustrated in Figs. 3 and 4, and their components are elaborated below.

A. Semantic Encoding of Protocol Standards

At the foundation of both proposed architectures lies the semantic encoding of protocol standards, which transforms unstructured textual specifications into structured, machine-readable representations suitable for retrieval and compliance reasoning. The objective is to map natural language descriptions of IPv6 RFC clauses into high-dimensional embedding vectors that preserve semantic meaning and contextual dependencies. This enables efficient similarity computation between packet-level compliance queries and relevant RFC passages, thereby forming the basis of the retrieval-augmented compliance framework [16].

1) Document Preprocessing and Chunking: All standard documents, including IPv6 RFCs and their extensions, are first preprocessed to extract textual content and remove extraneous formatting artifacts. The text is then segmented into semantically coherent units, or *chunks*, each limited to a maximum of 512 tokens to align with transformer model input constraints while retaining local context. Formally, for a given RFC document D, we obtain a sequence of contiguous chunks

$$D = \{C_1, C_2, \ldots, C_N\} \tag{1}$$

where each C_i denotes the ith chunk and N is the total number of extracted segments. This chunking strategy preserves semantic integrity while ensuring tractable embedding computation.

2) Transformer-Based Embeddings: Each chunk C_i is projected into a high-dimensional semantic space using a sentence transformer, a pretrained model optimized for semantic search tasks. The embedding process can be expressed as

$$\mathbf{v}_i = \text{TransformerEncoder}(C_i) \tag{2}$$

where $\mathbf{v}_i \in \mathbb{R}^{768}$ is the resulting embedding vector for chunk C_i. The encoder leverages multihead self-attention and pooling operations (e.g., mean pooling) to integrate contextual dependencies across sentences.

At each transformer layer l, the hidden state representation $\mathbf{H}^{(l)}$ is computed via self-attention

$$\mathbf{H}^{(l)} = \text{SelfAttention}(\mathbf{H}^{(l-1)}) \tag{3}$$

where the self-attention mechanism itself is defined as

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{4}$$

with query, key, and value matrices Q, K, and V, and d_k representing the dimensionality of the queries and keys. The final embedding $\mathbf{v}_i$ is taken from the output of the last transformer layer after pooling.

3) Semantic Similarity: The embeddings are designed such that semantic similarity between two chunks corresponds closely to their vector similarity. This is measured using cosine similarity

$$\text{sim}(\mathbf{v}_i, \mathbf{v}_j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\|\|\mathbf{v}_j\|} \tag{5}$$

where a higher similarity score reflects stronger semantic relatedness between chunks C_i and C_j.

These embeddings, once generated, are indexed into a vector database for efficient retrieval, as discussed in Section III-B. The semantic encoding step thus provides the critical bridge between natural-language protocol standards and automated compliance verification, enabling both architectures to operate on a unified semantic representation of IPv6 specifications.

B. Vector Database and Semantic Retrieval

To enable efficient semantic access to IPv6 protocol specifications, the high-dimensional embeddings produced during

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

semantic encoding are organized and indexed within a persistent vector database. Given the computational intractability of linear similarity search in large embedding spaces, approximate nearest neighbor (ANN) indexing is employed to achieve both scalability and low-latency retrieval. Specifically, we adopt the hierarchical navigable small world (HNSW) graph [36], a graph-based ANN structure that is widely recognized for its robustness in high-dimensional search tasks.

1) HNSW Graph Indexing: Each chunk embedding derived from the RFC corpus is stored as a node in the HNSW graph. Edges between nodes are induced by cosine distance, so semantically proximate embeddings form densely connected local neighborhoods. Formally, let

$$V = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_N\}, \quad \mathbf{v}_i \in \mathbb{R}^{768}$$

denote the set of all stored embeddings, and let $d(\cdot,\cdot)$ be cosine distance. For a given node $\mathbf{v}_i$, let $\mathbf{v}_{(k)}$ denote its kth nearest neighbor in $V \setminus \{\mathbf{v}_i\}$ under $d(\cdot,\cdot)$. We define the k-radius neighborhood of $\mathbf{v}_i$ as

$$U_k(\mathbf{v}_i) = \left\{\mathbf{v}_j \in V \setminus \{\mathbf{v}_i\} \;\middle|\; d(\mathbf{v}_i, \mathbf{v}_j) \le d(\mathbf{v}_i, \mathbf{v}_{(k)})\right\} \quad (6)$$

which captures all embeddings at least as close to $\mathbf{v}_i$ as its kth nearest neighbor (i.e., a distance-thresholded local neighborhood). In HNSW, nodes are inserted probabilistically into multiple hierarchical layers: higher layers provide sparse long-range links that act as navigational shortcuts, while lower layers preserve dense local connectivity. This hierarchical structure enables efficient approximate nearest-neighbor search with low query latency in high-dimensional embedding spaces.

2) Query-Time Retrieval: At query time, a compliance-related input, such as a packet description or a field-specific question, is encoded into an embedding vector $\mathbf{q} \in \mathbb{R}^{768}$ using the same sentence transformer employed during document encoding. The retrieval objective is to identify the top-k nearest neighbors to $\mathbf{q}$ according to cosine similarity

$$R(\mathbf{q}) = \arg \max_{\mathbf{v}_i \in V,\, |R|=k} \text{sim}(\mathbf{q}, \mathbf{v}_i) \quad (7)$$

where similarity is defined as

$$\text{sim}(\mathbf{q}, \mathbf{v}_i) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\|\|\mathbf{v}_i\|}. \quad (8)$$

To ensure retrieval precision, we apply a similarity threshold τ, such that only embeddings sufficiently close to the query are retained

$$R_\tau(\mathbf{q}) = \{\mathbf{v}_i \in R(\mathbf{q}) \mid \text{sim}(\mathbf{q}, \mathbf{v}_i) \ge \tau\}. \quad (9)$$

This empirically chosen threshold was found to effectively suppress spurious matches while retaining all semantically relevant IPv6 clauses. Fig. 1 illustrates the process of ANN search combined with HNSW algorithm.

3) Reconstructed Sections: Since individual chunks often represent only partial clauses of the standard, retrieval results from the same source document are concatenated to form *reconstructed sections*. These reconstructed sections provide semantically coherent context windows that preserve both local meaning and broader document continuity. They serve as the primary knowledge units for downstream compliance reasoning, ensuring that generative agents evaluate packet validity against consistent and contextually rich evidence. Compliance classification assumes the full packet record is available; if fields are missing, some rules become unverifiable and the same accuracy cannot be guaranteed. RFC chunking does not weaken reasoning because retrieved chunks are merged into reconstructed sections, and the framework retrieves all relevant sections for each query by scaling agent deployment to the number of sections returned, ensuring clause-complete (not fragmentary) context. Compared with traditional keyword-based search, semantic retrieval via HNSW indexing provides significant advantages. Keyword systems fail when terminology varies (e.g., "payload length" versus "size of data field"), while embedding-based retrieval aligns semantically equivalent but lexically distinct phrases. Moreover, the HNSW structure ensures efficient scalability, reducing query latency to subsecond levels even for large-scale document collections.

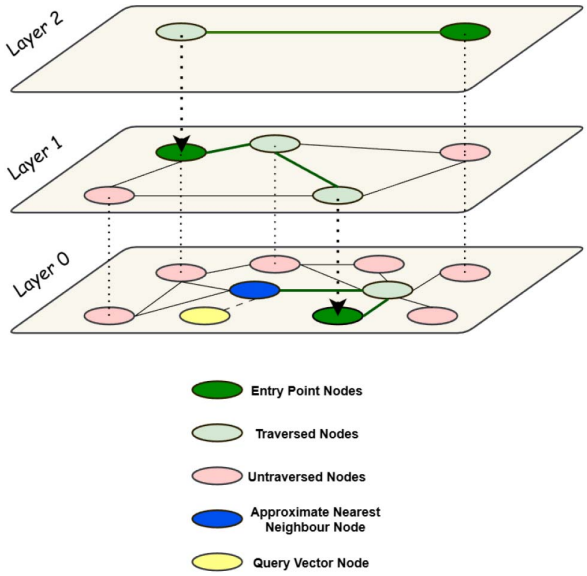


Fig. 1. Visualization of ANN search with the hierarchical navigable small world (HNSW) algorithm.

4) Model Families and Architectural Diversity: The reconstructed sections produced by semantic retrieval serve as the authoritative context for all downstream reasoning. Each section aggregates contiguous, clause-coherent text from a single RFC, preserving local semantics and cross-references. We pass these sections verbatim, along with lightweight metadata identifying the source RFC and clause boundaries to the generative agents. We evaluate distinct LLM families to probe how architectural choices affect retrieval grounding, clause interpretation, and downstream agent behavior. Specifically, we consider OpenAI's GPT series, Meta's Llama models, Anthropic's Claude models, Google's Gemini family, Alibaba's Qwen series, and open-source families including Mistral AI's Mixtral and Mistral models and the Gemma models developed through the Google–NVIDIA collaboration. As illustrated in Fig. 2, Llama variants

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

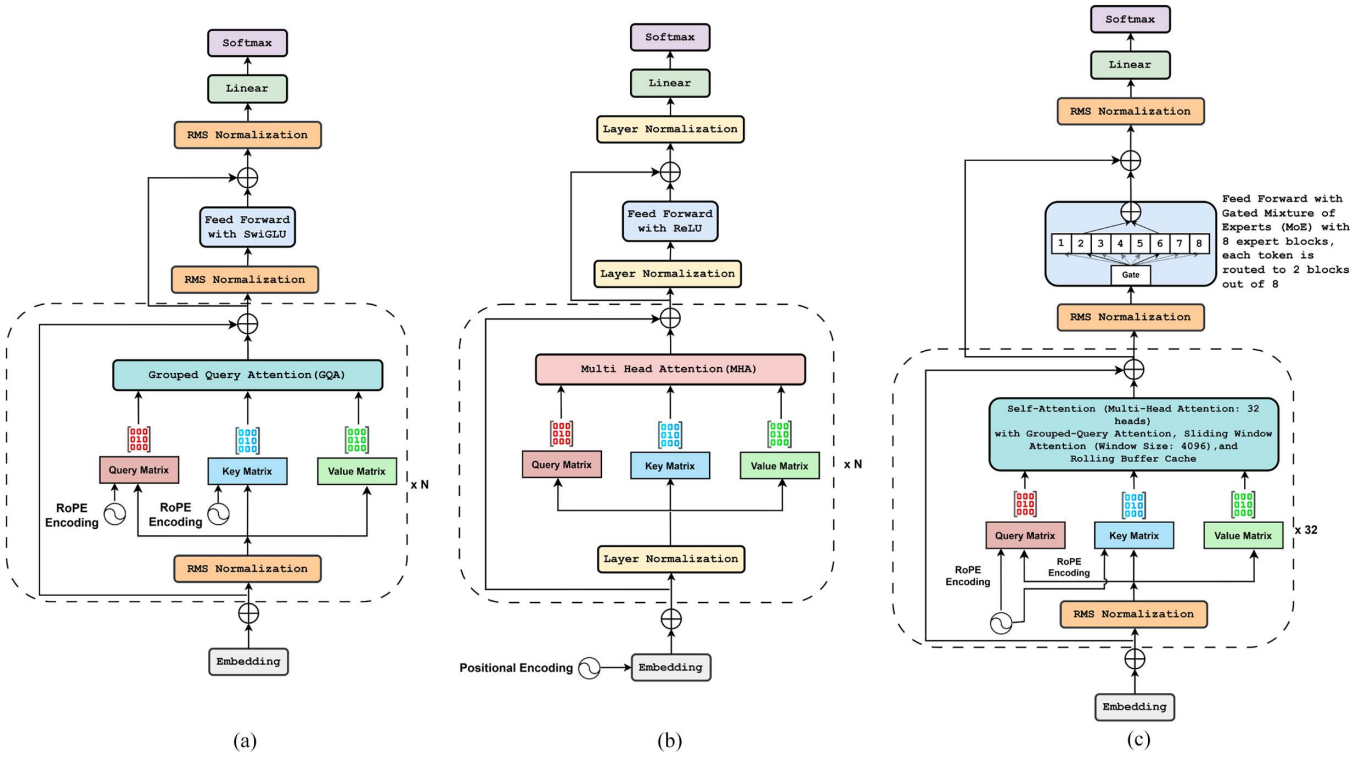


Fig. 2. Basic architecture diagram of: (a) llama; (b) GPT; and (c) mixtral 8x7b.

employ grouped-query attention (GQA), RMS normalization, and rotary positional embeddings (RoPE); GPT-family models use multihead attention (MHA), layer normalization, and learned/rotary positional schemes; and Mixtral augments transformer layers with a gated mixture-of-experts (MoE) router and grouped/sliding-window attention. This architectural diversity materially impacts reasoning style, long-context handling, and error modes. In the experiments that follow, we therefore adopt single-model ensembles, instantiating all agents for a given run from the same backbone, to enable controlled, within-model comparisons of section-level judgments and debate dynamics.

C. Dual-Architecture Generative Compliance Evaluation

To operationalize the semantic retrieval outputs for IPv6 compliance verification, we designed two complementary generative architectures. Both consume reconstructed protocol sections from the vector database and apply LLMs to evaluate compliance, but they differ fundamentally in how the packet evaluation is performed. In the first architecture, each LLM agent is assigned to a specific protocol section and independently assesses individual IPv6 packets with respect to that section, issuing a section-specific verdict and rationale. In contrast, the second architecture does not expose LLMs to packet data directly; instead, agents extract executable compliance rules from their assigned sections, which are subsequently composed into a unified Python script. This script is then applied to the dataset for high-speed, rule-based verification. Together, these two approaches highlight complementary strengths, section-level interpretability and deliberation in the first case, and dataset-wide automation and latency reduction in the second, thereby mapping out the trade-off space between transparency and efficiency in generative compliance systems.

1) Architecture A (Explainability Mode): The retrieved protocol specification segments obtained through semantic retrieval form the contextual foundation for our generative multiagent compliance verification system. In this approach, each protocol compliance query initiates the parallel deployment of multiple generative AI agents, each tasked with independently evaluating the provided semantic context to determine IPv6 packet compliance. Rather than integrating heterogeneous generative models within a single ensemble, our system is organized as *single-model* ensembles: each experiment instantiates all agents from the same state-of-the-art LLM, enabling a clean, within-architecture assessment of interpretive strengths, biases, and consistency across sections. Fig. 3 illustrates the framework for AICCE in explainability mode.

For a given packet Q and reconstructed section C_i, a dedicated agent A_i receives the section text, a curated file of general IPv6 rules R_g, and the full packet features. Each agent returns a binary verdict with rationale in response to: *"Does the packet satisfy all required compliance properties under this section?"* Formally, the initial assessment is

$$V_i^{(0)} = \mathrm{LLM}_i(Q, C_i, R_g) \tag{10}$$

where LLM_i denotes an instance of the chosen backbone model. Inference settings are held fixed for comparability, and all agents for a packet run concurrently via a thread-based executor.

If all section agents return `Yes`, the packet is accepted; otherwise, a structured debate is triggered to resolve conflicts

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

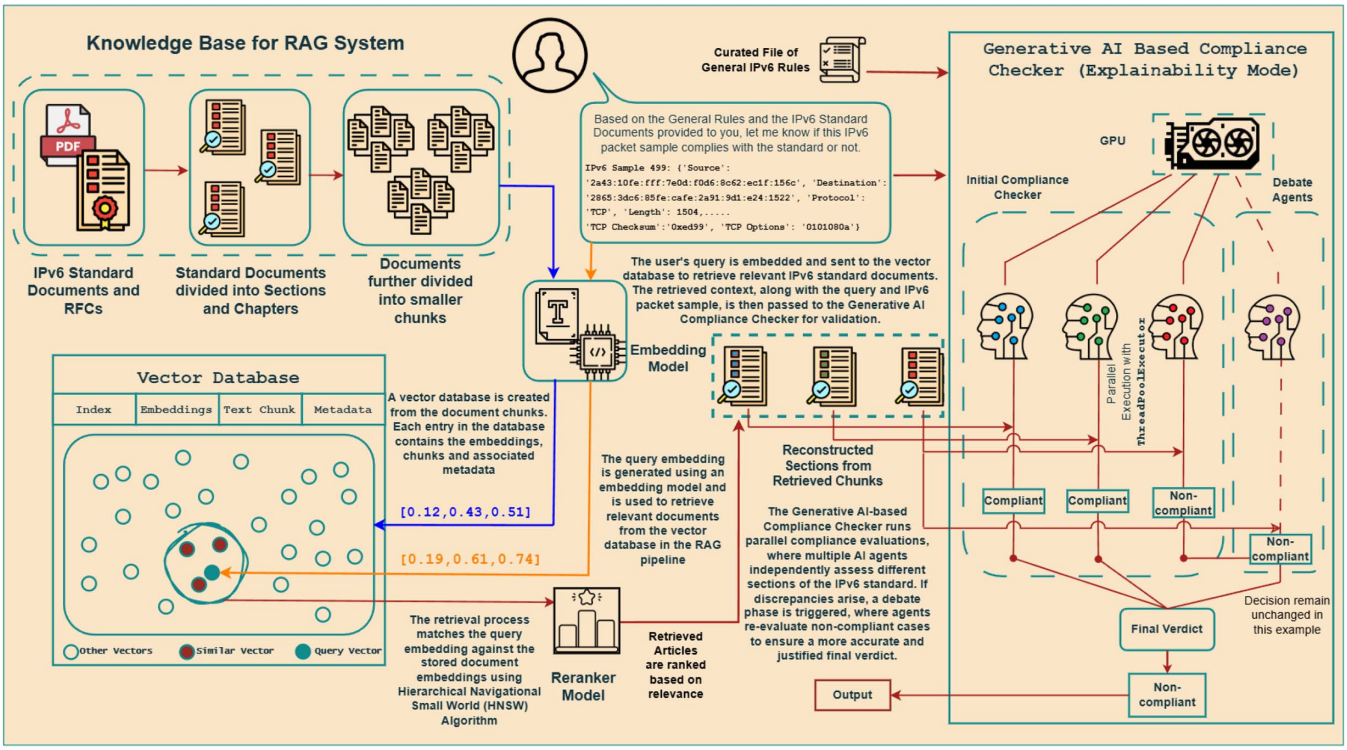


Fig. 3. AICCE Framework in explainability mode.

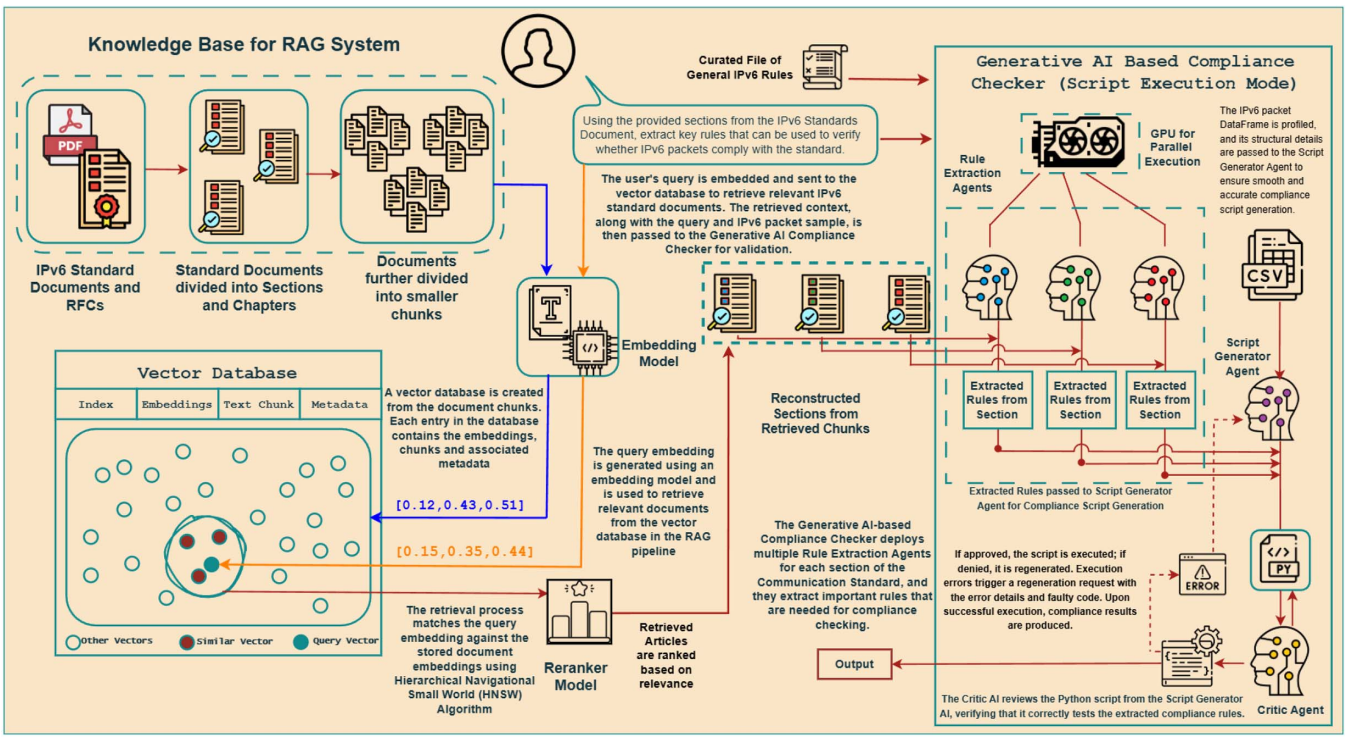


Fig. 4. AICCE Framework in script execution mode.

and surface clause-level justifications. At debate iteration $t \geq 1$, each agent revises its verdict after being shown peer rationales from round $t-1$

$$V_i^{(t)} = \mathrm{LLM}_i\big(Q, C_i, R_g, \{V_j^{(t-1)}\}_{j \neq i}\big). \tag{11}$$

Debate rounds for each LLM execute in parallel and are capped at five to bound latency; convergence to unanimous compliance leads to acceptance, else the packet is declared noncompliant. Inspired by recent advancements in multi-agent generative AI research [18], this design yields clause-level

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

interpretability and systematically mitigates single-agent brittleness by exposing and adjudicating disagreements through informed counterargumentation.

2) Architecture B: Script Execution Mode: The second architecture pursues a complementary objective: to minimize decision latency while retaining auditability through explicit, executable checks. It does so by translating textual clauses into Python-level rules and running them directly over the dataset.

Each reconstructed section is passed to a *section rule extraction* agent that reads the section text and R_g, cross-checks available fields in the DataFrame schema, and emits only actionable rules involving present columns. Rules are expressed as single-line predicates. A simplified example could be

$$\texttt{packet["Version"] == 6}$$
$$\texttt{packet["HopLimit"]} < 256. \tag{12}$$

A *script generator* aggregates the rules into a compliance checking function that computes a Boolean column per rule and a final `Compliant` column (logical AND over all rules). A *critic* reviews the script against the rules and schema; if deficiencies are found, a refinement loop ensues

$$S^{(t+1)} = \text{ScriptGen}\big(R,\ \text{Critic}(S^{(t)}, R)\big) \tag{13}$$

which repeats until the critic returns `APPROVED` or a turn limit is reached. If runtime errors occur, the traceback is fed back to script generator for targeted repair. Once approved, the script executes over the full dataset, yielding a verdict per row plus rule-level diagnostics. This pipeline attains sub-second per-packet latency (approximately 0.8 s on average), offering substantial throughput gains versus debate. Fig. 4 illustrates the framework for AICCE in script execution mode.

3) Parallelism, Coverage, and Aggregation: In both architectures, the number of concurrently instantiated units adapts to the number of reconstructed sections returned by retrieval, ensuring each clause is evaluated in context. Execution is parallelized using a thread pool to reduce wall-clock latency. Architecture A aggregates section-level verdicts after debate, requiring unanimity for compliance; Architecture B aggregates rule columns via logical conjunction to form row-level labels. The former yields rich rationales and cross-checked judgments, while the latter provides a sparse, auditable rule matrix suited to high-throughput screening.

4) Comparison of Architectures: Architecture A foregrounds interpretability, thorough section-by-section reasoning, and explicit adjudication of disagreement; Architecture B prioritizes speed and deterministic execution via dataset-aware rules. Table II summarizes the principal trade-offs.

TABLE II
COMPARISON OF FRAMEWORK ARCHITECTURES

Aspect	Architecture A	Architecture B
Primary focus	Interpretability, coverage	Speed, automation
Multiagent usage	Per section & per packet	Per section
Debate mechanism	Yes	Yes
Granularity	Section-level judgments	Rule-level execution
Output	Rationale + verdict	Rule matrix + verdict
Exec speed	~ 2 s/packet	~ 0.8 s/packet

TABLE III
VIOLATION TYPES BY DETECTION DIFFICULTY

Violation Type	Difficulty	Occurrences
IPv6 version violation	Trivial	52
Length field violation	Moderate	54
IPv6 address violation	Trivial	54
Hop limit violation	Moderate	52
TCP/UDP/ICMPv6 overlap	Challenging	49
Flow label violation	Moderate	54
DSCP field violation	Moderate	63
ECN field violation	Moderate	49
Protocol mismatch violation	Challenging	52
Next header field violation	Challenging	46
ICMPv6 code violation	Moderate	53

IV. EXPERIMENTS

This section describes the experimental evaluation conducted to assess the performance of the proposed IPv6 compliance verification framework. The experimental protocol begins with an exploratory data analysis (EDA) to clearly characterize the dataset, including its origins, data integrity considerations, and noncompliance anomalies introduced for comprehensive testing.

A. Exploratory Data Analysis

The dataset utilized for evaluating IPv6 compliance detection was sourced from the Center for Applied Internet Data Analysis (CAIDA). Specifically, the Day 1 IPv6 trace dataset was selected due to its high data integrity, minimal corruption rate, and representative coverage of real-world IPv6 traffic. CAIDA's datasets are widely recognized and employed in network research due to their comprehensive and clean capture of genuine Internet traffic patterns, thereby ensuring realistic and reproducible experimental conditions.

Initially, we randomly sampled 1500 packets from the Day 1 dataset, and these samples were all considered standards-compliant upon selection.

To rigorously evaluate the framework's ability to detect protocol anomalies, a subset of noncompliant samples was generated through intentional violations. Specifically, 300 randomly selected compliant samples were modified by applying one to three distinct violations, thereby rendering these packets noncompliant according to IPv6 standards. Due to the randomized violation selection, the total number of violations exceeds the number of noncompliant samples, leading to richer testing scenarios.

Table III summarizes the frequency of each introduced violation across the 300 noncompliant samples. Although exactly 300 samples were modified, the total number of individual violations introduced was 578, given that multiple violations were often applied per sample.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

Each violation was further categorized by detection difficulty into three classes: 1) trivial; 2) moderate; and 3) challenging, based on how easily an anomaly could be identified or inferred through simple rule-based checks versus requiring sophisticated semantic reasoning. Table III provides this categorization and the frequency of each violation type per category. The details of each violation type have been discussed in Appendix C.

This EDA clarifies the structure and challenges posed by the constructed dataset. The deliberate injection of diverse and multiple violations per sample generates a comprehensive set of scenarios that thoroughly stress-test the robustness and accuracy of the proposed compliance detection system. By explicitly categorizing these violations, the dataset provides both straightforward and nuanced challenges for system evaluation, thus laying a solid foundation for experimental validation.

B. Experiment Settings

All evaluations were executed under a single, deterministic pipeline to ensure comparability across large–language–model (LLM) ensembles and to facilitate full reproducibility. Identical code, prompts, and hyper-parameter values were used for every LLM provider.

1) Vector–Database Construction: Official IPv6 RFCs and relevant addenda were downloaded in PDF form and automatically segmented into sentence-level chunks with an upper bound of 512 tokens per chunk. Each chunk was embedded with a 32-layer sentence-transformer optimised for semantic search [16]. The resulting $d = 768$-dimensional vectors were persisted in a ChromaDB collection, indexed with an HNSW graph (`hnsw:space = cosine`) for subsecond approximate-nearest-neighbour lookup [36]. Every record stores: 1) the raw text; 2) its embedding; and 3) a single JSON metadata key `document_name`, enabling rapid source tracing during error analysis.

2) Semantic Retrieval: At run time an ad-hoc query embedding is produced with the same transformer and matched against at most $k = 50$ neighbours. A similarity threshold of $\tau = 0.60$ is applied to filter low-relevance matches; this value was chosen empirically as the smallest threshold that suppressed obvious false positives while retaining all ground-truth IPv6 clauses in a validation sweep. Retrieved chunks belonging to the same RFC are concatenated into a *reconstructed section*. Ranking is solely by HNSW cosine distance, and a lightweight metadata-based reordering is performed in which chunks are grouped and prioritized by their `document_name` field. This ensures that related segments from the same RFC are surfaced together, preserving clause continuity and contextual integrity.

3) Packet-Level Compliance Agents (Architecture A): For architecture A (*explainability mode*), every reconstructed section spawns an independent compliance agent. Each agent is a single LLM call that receives: 1) the reconstructed section; 2) the global IPv6 general-rules file; and 3) the packet under inspection encoded as a JSON dictionary. All LLMs are executed with fixed inference settings (`temperature` $= 0.7$, `top_p` $= 1.0$, provider-default `top_k`). Agents return an explicit `Yes`/`No`/`Maybe` verdict plus a one-sentence rationale. All section agents for a given packet are run concurrently via Python's `ThreadPoolExecutor` (`max_workers`=8).

4) Multiagent Debate (Architecture A): When an agent expresses uncertainty in its initial verdict, or when at least two section-specific agents disagree, a structured debate phase is triggered. For each disputed section, a new "Debate Agent", implemented as a single LLM call with the same inference hyper-parameters, is prompted with the uncertain or conflicting rationales, together with the original section, the general rules, and the packet. The debate agent must then produce a refreshed `Yes`/`No` decision, prefixed accordingly, followed by a concise justification. Debate rounds are capped at five at the LLM level; since each packet is checked across multiple sections, multiple debates may be invoked per packet. A packet is labelled *compliant* only if *all* section agents agree after the final debate round; otherwise, it is declared *noncompliant*. In practice, most debates converge in ≤ 2 rounds, though the framework allows up to five rounds per disputed section.

5) Rule Extraction Agents (Architecture B): For Architecture B, each reconstructed section is processed by a dedicated *rule extraction agent*. These agents operate independently and map textual clauses from the RFC sections and global IPv6 compliance rules onto executable logical conditions. To maintain alignment with the dataset, rules that reference fields absent from the packet schema are automatically discarded. Each agent is a single LLM call configured with fixed inference settings (`temperature` $= 0.7$, `top_p` $= 1.0$, provider-default `top_k`), ensuring consistency across rule extraction tasks.

6) Multiagent Debate (Architecture B): In Architecture B (*script execution mode*), compliance is determined through executable rule scripts rather than direct packet-level LLM reasoning. Here, the *script generator agent* translates extracted rules into a Python function, which is then reviewed by a *critic agent*. If flaws are found, an iterative debate cycle is launched: the script generator produces a revised script $S^{(t+1)}$ conditioned on the Critic's feedback of $S^{(t)}$. Unlike Architecture A, where debate is capped at five rounds per section, the script–critic cycle is permitted up to 20 iterations to ensure a fully executable and correct script. Runtime errors encountered during execution are also looped back into this cycle until either a valid script is produced or the retry cap is reached. Prompt samples for both architectures A and B have been discussed in Appendix D. The effect of multiagent debate for reasoning has been discussed in Appendix E with important examples for both architectures.

7) LLM Providers and Access Layer: All LLMs were accessed exclusively through production-ready APIs. Identical inference parameters were enforced across all experiments with no prompt tuning, ensuring strict comparability across providers.

A total of 16 state-of-the-art LLMs were evaluated. Table IV provides the complete mapping of model, provider, and API/access layer.

This per-model specification ensures clarity regarding both provider and execution environment, while covering a diverse set of architectures ranging from dense transformers (GPT, Claude, Gemini) to mixture-of-experts (Mixtral) and lightweight transformer variants (Gemma).

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
LLM PROVIDERS, MODELS, AND ACCESS LAYERS USED IN EXPERIMENTS

Provider	Model(s)	API/Access Layer
OPENAI	GPT-4o o1 GPT-4-Turbo GPT-4o-mini GPT-3.5-Turbo	OpenAI API [37]
META	Llama 3.3-70B Llama 3.2-11B Llama 3.1-8B	Groq API [38]
ANTHROPIC	Claude 3.5 Sonnet Claude 4.5 Sonnet	Claude API [39]
GOOGLE	Gemini 2.0 Flash Gemini 2.5 Flash	Gemini API [40]
ALIBABA CLOUD	Qwen-3-32B	Groq API [38]
MISTRAL AI	Mistral 7B	HuggingFaceHub [41]
NVIDIA--GOOGLE	Gemma 3-27B Gemma 2-9B	HuggingFaceHub HuggingFaceHub

C. Ablation Testing Settings

To evaluate the contribution of the debate mechanism to compliance accuracy, we conducted ablation experiments using only architecture A (explainability mode) but with the *multiagent debate phase disabled*. In this setting, each compliance agent independently assessed its assigned section of the RFC, applied the general IPv6 rules to the input packet, and produced a binary `Yes`/`No` verdict accompanied by a short rationale. However, unlike the full framework, no subsequent debate or cross-agent reconciliation was permitted: the initial verdict of each agent was treated as final.

V. RESULTS

In this section, we present a comprehensive evaluation of our compliance classification framework across multiple architectural modes and experimental settings. The goal is to assess not only the raw performance of individual LLMs but also the contributions of multiagent debate and rule-execution pipelines to overall system accuracy, robustness, and efficiency. Results are reported using standard metrics of accuracy, F1-score, and latency, supplemented by detailed analysis of model reasoning quality and system-level behavior.

We first conduct an ablation study to isolate the role of the multiagent debate mechanism by evaluating architecture A in *explainability mode* with the debate component disabled. This baseline reveals the raw ability of different models to classify compliance based solely on their initial reasoning.

Next, we reintroduce the multiagent debate mechanism in Architecture A, measuring how collaborative reasoning improves decision quality. In addition to accuracy, F1, and latency, we also report the number of debate rounds triggered per model. This provides insight into how frequently models disagreed, and how debate dynamics resolved ambiguities. Special attention is given to chain-of-thought (CoT) models [14], which benefit disproportionately from explainability-driven modes.

We then evaluate architecture B, where compliance is determined through executable rule scripts rather than direct LLM classification. This mode emphasizes reproducibility and correctness guarantees: we report not only performance metrics but also script retry counts and error recovery rates, which highlight the strengths and limitations of the script–critic loop.

Finally, we present a comparative analysis, where we synthesize results across both architectures and benchmark our framework against classical machine learning and deep learning baselines. This comparison situates our system in the broader compliance verification landscape and underscores the advantages of agentic, explainability-driven approaches.

A. Ablation Testing Results

To quantify the impact of the multiagent debate mechanism, we first evaluate architecture A in *explainability mode* without the debate component (ablation testing). We then reintroduce the debate mechanism and compare results directly. Figs. 5–7 present side-by-side comparisons of classification accuracy, F1-score, and latency, respectively. In addition, Fig. 8 reports the frequency with which debate phases were triggered, illustrating the role of collaborative reasoning in improving verdict quality.

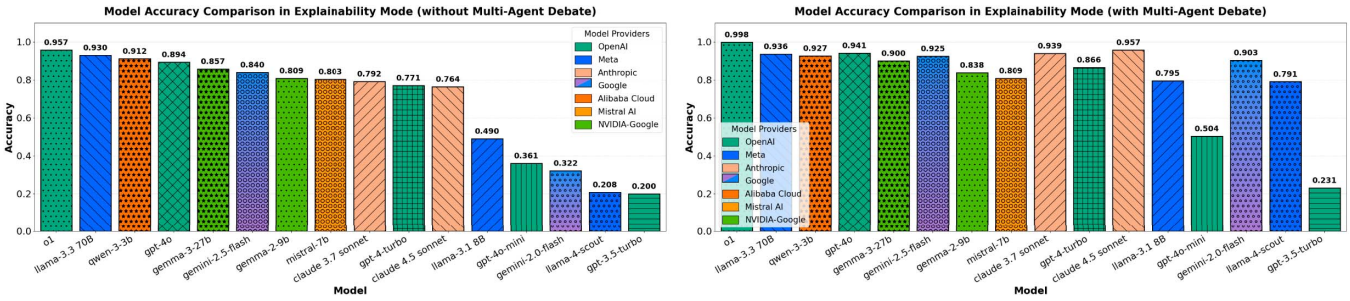
Models with explicit CoT capabilities, such as `o1` and `Llama-3.3-70B`, perform strongly even in the ablation setting because their step-by-step reasoning supports more reliable single-pass compliance judgments. Their higher latency is primarily driven by verbose reasoning traces and detailed rationale generation. In contrast, without the multi-agent debate mechanism, models with weaker initial judgment or less stable reasoning cannot revise erroneous decisions, leading non-CoT and lighter-weight models to exhibit substantially lower accuracy and F1 and greater vulnerability to misclassification in the absence of a corrective process.

A closer examination of the models' explanations illustrates why CoT-based models such as `o1` outperforms the non-CoT approach. The CoT-based models consistently produce step-by-step rationales for its compliance verdicts.

For example, consider a packet where the `Protocol` field is set to ICMP, yet the record clearly contains TCP-related columns, and the Flow Label value is set to `0xb7cb4a` (decimal 12 035 274). The CoT-based model such as `o1`, produces the following detailed explanation.

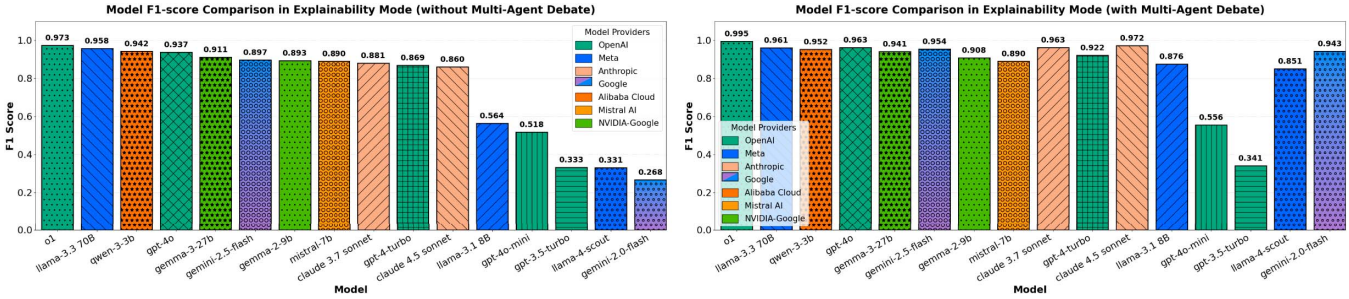
> **Verdict: Non-Compliant**
>
> **Rationale:**
> Step 1: The `Protocol` field is set to ICMP. According to RFC 8200, the `Protocol` field must correspond to the actual transport-layer header present in the packet.
> Step 2: The record contains TCP-related fields, which directly contradicts the ICMP setting, indicating a protocol mismatch.
> Step 3: The Flow Label is set to `0xb7cb4a` (12,035,274), which exceeds the 20-bit maximum defined in RFC 8200 (Section 6). Valid Flow Label values must be less than $2^{20} = 1{,}048{,}576$.
> Step 4: Since both conditions violate IPv6 compliance rules, the packet is classified as *Non-Compliant*.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.



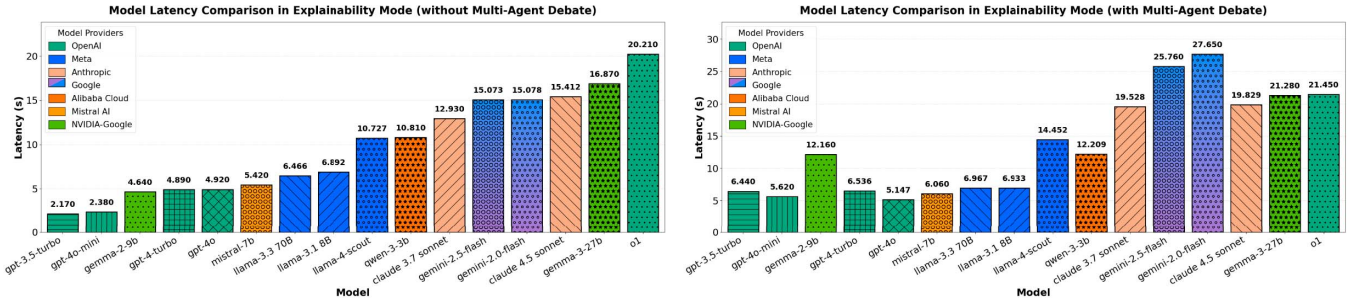
(a) (b)

Fig. 5. Model accuracy comparison under ablation versus debate-enabled settings for architecture A. Debate consistently improves accuracy across models, particularly for weaker baselines. (a) Without multiagent debate. (b) With multiagent debate.



(a) (b)

Fig. 6. F1-score comparison under ablation versus debate-enabled settings for architecture A. Debate reduces false positives and stabilizes precision–recall balance. (a) Without multiagent debate. (b) With multiagent debate.



(a) (b)

Fig. 7. Decision latency comparison under ablation versus debate-enabled settings for architecture A. As expected, enabling debate increases runtime, but accuracy gains justify the overhead. (a) Without MULTIAGENT debate. (b) With multiagent debate.

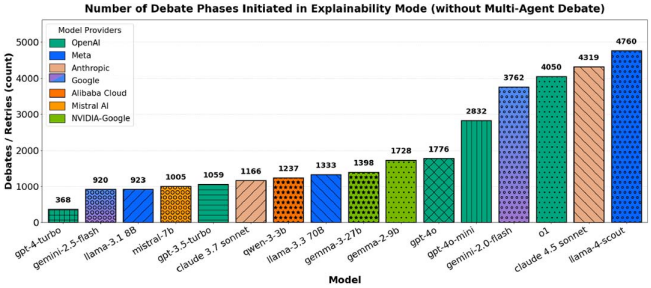


Fig. 8. Number of debate phases initiated per model in architecture A. Stronger CoT models required fewer debates, while weaker baselines triggered debates more frequently.

In contrast, a non-CoT baseline model such as GPT-3.5-Turbo, might return a shorter and less precise justification such as.

> **Verdict: Non-Compliant**
>
> **Rationale:** Flow Label exceeds the 20-bit maximum.

Here, the non CoT-based LLM flagged the flow label violation but entirely missed the protocol mismatch caused by the contradictory ICMP/TCP overlap, resulting in an incomplete analysis, although the compliance verdict was correct in this case.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

In some instances, the baseline even misclassified borderline compliant cases due to its lack of deeper analysis. By explicitly referring to the relevant criteria in its reasoning, `o1` not only offers greater transparency but also makes more accurate decisions. This example highlights how CoT reasoning improves both interpretability and performance in the absence of a debating mechanism.

B. Architecture A: Explainability Mode Results

When the multiagent debate mechanism is enabled, Architecture A exhibits a marked improvement in both accuracy and F1-score across nearly all evaluated models. Figs. 5–7 compare performance before and after the introduction of debate, while Fig. 8 shows the number of debate phases triggered, offering insight into how disagreement resolution influenced final verdicts.

Overall, multiagent debate functions as a corrective layer that most strongly benefits models prone to initial misjudgments or unstable reasoning. In particular, weaker baselines such as `Gemini-2.0-Flash`, `Llama-4-Scout`, and `Llama-3.1-8B` become substantially more competitive once debate is enabled, consistent with their high debate activity (i.e., frequent disagreement resolution over the evaluation set). Stronger models such as `o1`, `GPT-4o`, `Llama-3.3-70B`, and `Qwen-3-3B` exhibit smaller deltas, suggesting their single-pass reasoning is already near-optimal and debate mainly resolves a residual set of ambiguous cases.

At the same time, debate is not theoretically guaranteed to improve every model under every evaluation regime. Its impact depends on the base model's calibration, the relative strength of debating agents, the debate trigger frequency, and whether additional rounds introduce noise or over-correction on borderline cases. When a model is already near ceiling performance, a small number of debate-induced flips can shift the precision–recall balance and slightly reduce F1, especially in limited-scale evaluations; with larger evaluation sets, these effects typically stabilize and the net corrective benefit becomes more consistent.

Enabling debate increases latency, as expected, but the overhead is generally acceptable given the robustness and reliability gains. High-capacity models tend to incur moderate additional runtime, whereas some hosted APIs exhibit higher absolute latency. Overall, these results motivate architecture A when explainability and deliberative reliability are desired, while acknowledging that debate benefits are conditional rather than universal.

C. Architecture B: Script Execution Results

Architecture B adopts a fundamentally different approach by shifting from natural language verdict generation to executable compliance scripts. This design enforces stricter consistency, as the models must output structured code that is executed against packet attributes. Fig. 9(a)–(c) reports accuracy, F1-score, and latency for all models, while Fig. 9(d) shows the number of retries required when script generation initially failed.

Overall, the results show that architecture B is highly reliable for strong foundation models, which can consistently translate RFC clauses into executable compliance logic under the script-execution verifier. Models such as `o1`, `GPT-4o`, `GPT-4-Turbo`, `Llama-3.3-70B`, and the stronger `Claude` variants perform robustly, while several weaker models collapse; such as predicting a single class in an imbalanced setting or fail to produce usable scripts within the retry budget (e.g., `GPT-3.5-Turbo`, `Llama-4-Scout`, `Qwen-3-3B`, `Gemma-2-9B`, `Gemma-3-27B`, `Mistral-7B`), highlighting architecture B's sensitivity to structured code generation quality. These failures highlight the sensitivity of architecture B to structured code generation: once a model repeatedly emits syntactically incorrect or semantically incorrect scripts, the execution framework provides no opportunity for recovery beyond the retry budget.

Latency results further emphasize architecture B's advantage: because compliance is enforced by executing concise scripts instead of producing long explanations, decision times are typically subsecond across models, making this mode suitable for high-throughput settings. Retry statistics align with these trends, high-performing models generally require few retries, whereas weaker models frequently exhaust the retry budget or repeatedly generate executable but semantically incorrect scripts, demonstrating that the execution framework sharply separates models that can reliably produce correct rule logic from those that cannot.

In summary, the updated results confirm that architecture B offers an excellent accuracy–latency trade-off when applied to strong foundation models. Unlike explainability mode, where multi-agent debate is necessary to iteratively repair reasoning errors, script execution mode enforces correctness at the code level, substantially reducing both variance and runtime for capable models. At the same time, it introduces a sharper divide between models that can reliably produce precise, executable compliance rules and those that cannot, with weaker models collapsing to trivial or unusable behavior. This duality underscores both the promise and the challenge of script-based compliance verification in high-stakes interoperability settings.

D. Comparative Analysis

Table V contrasts AICCE against classical and deep baselines using accuracy, precision, recall, F1, and latency. For AICCE, we report only the best result within each mode [architecture A without multiagent debate (MAD), architecture A, and architecture B] in terms of accuracy; full per-model results are deferred. The classical machine learning and sequential baselines are trained on 1000 labeled samples and evaluated on a held-out test set of 500 samples, whereas all AICCE modes are evaluated in a zero-shot regime over the full 1500 sample corpus (train + test), with no task-specific training or fine-tuning required. In Table V, the best metric values across all systems are shown in bold, and the titles of our proposed AICCE systems are also bolded for clearer identification. The complete results for all AICCE configurations and LLMs across all modes appear in Appendix B.

Classical discriminative models (e.g., logistic regression, random forest, MLP) achieve respectable accuracy ($\approx$ 0.88–0.94) but exhibit pronounced precision–recall imbalances that depress F1 (e.g., SVM precision 0.9831 versus recall 0.58; Naive Bayes precision 1.00 versus recall 0.41). Tree ensembles with stronger recall (random forest, extra trees) improve F1

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

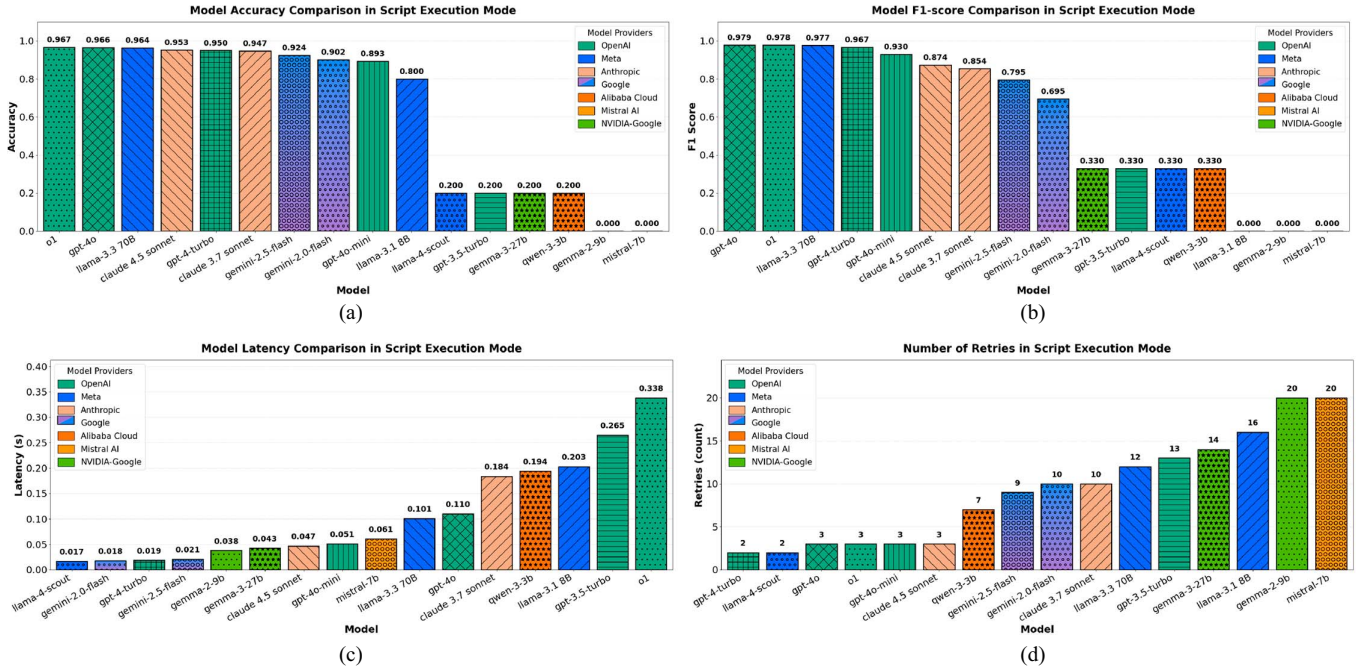


Fig. 9. Performance of models in script execution mode (architecture B). (a) Accuracy. (b) F1-score. (c) Latency. (d) Number of retries.

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT COMPLIANCE CHECKER SYSTEMS

System	Accuracy	Precision	Recall	F1-score	Latency	Explainability	Err. Recovery
Logistic Regression*	0.930	0.945	0.690	0.797	0.00035	No	No
Random Forest*	0.938	0.856	0.830	0.842	0.00086	No	No
Extra Trees*	0.928	0.833	0.800	0.816	0.00043	No	No
XGBoost*	0.560	0.298	0.890	0.447	0.00016	No	No
LightGBM*	0.500	0.265	0.850	0.404	0.00020	No	No
Support Vector Machine*	0.914	0.983	0.580	0.729	0.00029	No	No
Gradient Boosting*	0.850	0.603	0.730	0.660	0.00145	No	No
K-Nearest Neighbors*	0.888	0.958	0.460	0.622	**0.00001**	No	No
Naive Bayes*	0.882	**1.000**	0.410	0.582	**0.00001**	No	No
MLP Neural Net*	0.930	0.809	0.850	0.829	0.00233	No	No
BERT + LSTM*	0.840	**1.000**	0.200	0.333	0.365	No	No
Isolation Forest*	0.774	0.427	0.380	0.402	0.00070	No	No
AICCE Arch A (w/o MAD)	0.957	0.964	0.982	0.973	20.210	**Yes**	No
AICCE Arch A (with MAD)	**0.998**	**1.000**	**0.990**	**0.995**	21.450	**Yes**	**Yes**
AICCE Arch B	0.967	0.997	0.961	0.979	0.098	No	**Yes**

Note: * Trained on 1000 samples and tested on 500 samples.

into the 0.82–0.85 range, while boosting methods (XGBoost, LightGBM) underperform on this task. The sequence model (BERT+LSTM) attains perfect precision but very low recall (0.20), indicating systematic under-detection; the anomaly detector (Isolation Forest) similarly lags in both recall and F1. These baselines offer microsecond-scale latencies due to fixed feature pipelines, but none of these models offer explainability or error recovery. Implementation details for all compared systems are provided in Appendix A.

By contrast, AICCE delivers near-perfect performance across its operating modes while adding explicit compliance guarantees. In explainability mode without MAD (architecture A, ablation), the best-performing configuration achieves an accuracy of approximately 0.96, with F1-score around 0.97 and recall above 0.98, already surpassing the sequential baseline in both accuracy and F1. When the multiagent debate mechanism is enabled (architecture A with MAD), the strongest configuration reaches near-perfect classification quality with accuracy $\approx$ 0.998, F1 $\approx$ 0.995, and recall $\approx$ 0.99, at the cost of higher latency on the order of tens of seconds due to retrieval, explicit reasoning, and debate. This makes Architecture A particularly suitable for settings where rich natural-language justifications, human-in-the-loop review, and auditability are paramount (e.g., standards development, forensic post-incident analysis, or contested compliance decisions), since every verdict is accompanied by traceable, clause-grounded reasoning and, when needed, multiagent deliberation.

In script execution mode (architecture B), AICCE trades a small amount of peak accuracy for a substantial reduction in response time. The top configuration in this mode attains accuracy ≈ 0.97 and F1 ≈ 0.98 while reducing average decision time to well below one second, reflecting the efficiency of

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

executing synthesized rule scripts once clauses are compiled. Architecture B is therefore preferable in high-throughput, production monitoring scenarios (e.g., continuous protocol conformance screening or real-time gatekeeping), where thousands of messages must be processed per second and a concise pass/fail outcome is sufficient. In summary, architecture B offers the best accuracy–latency trade-off for automated pipelines, whereas architecture A is advantageous when maximum correctness, detailed explanations, and support for human oversight and evolving standards are more critical than raw throughput.

VI. Conclusion

A. Contributions and Discussion

This article presented AICCE, a generative framework for IPv6 standards compliance that uses retrieval-augmented generation and has two complimentary modes of operation. To reach auditable, clause-faithful judgments, Architecture A (explainability mode) combines clause-grounded, multiagent reasoning and structured discussion whereas Architecture B (script execution mode) assembles retrieved clauses into executable rules that provide predictable, low-latency compliance decisions. The AICCE achieves near-perfect accuracy and F1 in explainability mode (up to 0.998 accuracy and 0.995 F1) and high accuracy in script execution mode (up to 0.967 accuracy and 0.979 F1) across diverse LLM families by embedding IPv6 RFCs into a vectorized corpus and conditioning model behavior on retrieved passages. This results in transparent decisions whose evidentiary basis can be traced to specific standards text. Another noteworthy feature of the AICCE design is its zero-shot nature where we rely on standards-grounded retrieval and model-agnostic orchestration instead of task-specific fine-tuning. Together, the results of this study show that principled retrieval, agentic deliberation, and executable-rule synthesis can effectively close the gap between formal conformance testing and traditional tooling while maintaining interpretability; appropriate for certification and auditing.

B. Limitations

However, several limitations merit discussion. The chunking technique and retrieval of documents are crucial to AICCE's performance; missing or fragmented passages can impair reasoning and rule extraction, particularly for cross-referenced clauses. Occasionally, over-corrections suggest that human adjudication may still be necessary for edge cases, and debate helps to reduce, but not always eliminate, the ambiguities inherent in evolving RFCs. Significant model variance can be seen in weaker models, which can introduce latency and cost variability by failing deterministically at script synthesis or requiring numerous debate rounds. Even though the evaluation dataset is based on actual traces and is enhanced with a variety of violations, it is not possible to fully capture operational corner cases such as infrequent extension-header interactions, vendor actions, or hostile covert-channel designs.

C. Future Directions

Future work will focus on enhancing retrieval fidelity and knowledge curation by incorporating errata, drafts, and operational best practices, as well as learning re-rankers tailored to protocol discourse and measuring clause coverage with uncertainty estimates. We plan to integrate verifier–generator hybrids that combine debate with specialized verifiers, confidence calibration, and selective abstention to support human-in-the-loop review when evidence is weak or conflicting. A neuro-symbolic layer that compiles retrieved clauses into intermediate logical forms could provide stronger guarantees for temporal or cross-packet properties. Runtime orchestration can be made adaptive by routing samples between Architectures A and B according to estimated difficulty, explainability needs, and latency budgets, and by allocating debate or refinement iterations dynamically. With the help of sandboxed execution and policy guardrails, robustness will be put to the test against adversarial packets, parser-evasion strategies, and retrieval or prompt poisoning. To promote reproducible research in standards-constrained, clause-faithful reasoning, we also hope to expand the framework to other protocols and regulated domains, support multilingual standards, create distilled/on-device variants for edge settings, and publish benchmarks and audit artifacts annotated with clauses.

To summarize, AICCE shows that executable rule synthesis combined with zero-shot, retrieval-grounded multi-agent reasoning can provide high-accuracy, auditable IPv6 compliance at realistic latencies. The framework provides the basis for reliable AI-driven conformance auditing across contemporary networking and related regulated ecosystems by centering the decision-making process around standards text.

References

[1] D. S. E. Deering and B. Hinden, "Internet protocol, version 6 (IPv6) specification," RFC 8200, Jul. 2017. [Online]. Available: https://www.rfc-editor.org/info/rfc8200

[2] D. S. E. Deering and B. Hinden, "IP Version 6 addressing archit.," RFC 4291, Feb. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4291

[3] S. Deering and R. Hinden, "Internet protocol, version 6 (ipv6) specification," RFC 2460, 1998. [Online]. Available: https://www.rfc-editor.org/info/rfc2460

[4] A. Dua, V. Jindal, and P. Bedi, "Detecting and locating storage-based covert channels in internet protocol version 6," *IEEE Access*, vol. 10, pp. 110661–110675, 2022.

[5] A. Houmansadr, C. Brubaker, and V. Shmatikov, "The parrot is dead: Observing unobservable network communications," in *Proc. IEEE Symp. Security Privacy,* 2013, pp. 65–79.

[6] M. W. U. Rahman et al., "AI/ML based detection and categorization of covert communication in ipv6 network," 2025, *arXiv:2501.10627*.

[7] M. Roesch et al., "SNORT: Lightweight intrusion detection for networks," in *Proc. LISA*, vol. 99, no. 1, 1999, pp. 229–238.

[8] Open Information Security Foundation, "Suricata: Open Source IDS/IPS/NSM Engine," Open Inf. Secur. Found., 2024. [Online]. Available: https://suricata.io/

[9] G. J. Holzmann, "The model checker SPIN," *IEEE Trans. Softw. Eng.*, vol. 23, no. 5, pp. 279–295, May 1997.

[10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The tamarin prover for the symbolic analysis of security protocols," in *Proc. 25th Int. Conf. Comput. Aided Verification (CAV)*, 2013, pp. 696–701.

[11] M. Sutton, A. Greene, and P. Amini, *Fuzzing: Brute Force Vulnerability Discovery*, London, U.K.: Pearson Educ., 2007.

[12] T. Brown et al., "Language models are few-shot learners," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 1877–1901, Dec. 2020.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

[13] R. Bommasani et al., "On the opportunities and risks of foundation models," 2021, *arXiv:2108.07258*.
[14] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 24824–24837, Nov. 2022.
[15] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, Dec. 2020.
[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Nat. Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
[17] G. Irving, P. Christiano, and D. Amodei, "AI safety via debate," 2018, *arXiv:1805.00899*.
[18] M. W. U. Rahman, R. Nevarez, L. T. Mim, and S. Hariri, "Multi-agent actor-critic generative AI for query resolution and analysis," *IEEE Trans. Artif. Intell.*, vol. 6, no. 8, pp. 2226–2240, Aug. 2025.
[19] K. Sen, M. Viswanathan, and G. Agha, "On statistical model checking of stochastic systems," in *Proc. Int. Conf. Comput. Aided Verification*, Springer, 2005, pp. 266–280.
[20] H. L. Younes and R. G. Simmons, "Probabilistic verification of discrete event systems using acceptance sampling," in *Proc. Int. Conf. Comput. Aided Verification*, Springer, 2002, pp. 223–235.
[21] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and Z. Wang, "Time for statistical model checking of real-time systems," in *Proc. Int. Conf. Comput. Aided Verification*, Springer, 2011, pp. 349–355.
[22] M. Kwiatkowska, G. Norman, and D. Parker, "Prism 4.0: Verification of probabilistic real-time systems," in *Proc. Int. Conf. Comput. Aided Verification*, Springer, 2011, pp. 585–591.
[23] J. Pereyda, "Boofuzz: A Protocol Fuzzing Framework." Accessed: Apr. 19, 2024. [Online]. Available: https://github.com/jtpereyda/boofuzz
[24] M. Zalewski, "American fuzzy lop (AFL)." 2014. [Online]. Available: https://lcamtuf.coredump.cx/afl/
[25] A. Cooper, F. Gont, and D. Thaler, "Security and privacy considerations for IPV6 address generation mechanisms," Tech. Rep., 2016.
[26] "Internet protocol," RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/info/rfc791
[27] J. Czyz, M. Allman, J. Zhang, S. Iekel-Johnson, E. Osterweil, and M. Bailey, "Measuring IPV6 adoption," in *Proc. ACM Conf. SIGCOMM*, 2014, pp. 87–98.
[28] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed., NJ, USA: John Wiley & Sons, 2009.
[29] J. Konečnỳ, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.
[30] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Reviews)*, vol. 38, no. 2, pp. 156–172, Mar. 2008.
[31] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. Int. Conf. Neural Netw. (ICNN), vol*. 4, Piscataway, NJ, USA: IEEE Press, 1995, pp. 1942–1948.
[32] M. Dorigo, M. Birattari, and T. Stutzle, "Ant colony optimization," *IEEE Comput. Intell. Mag.*, vol. 1, no. 4, pp. 28–39, Aug. 2006.
[33] S. Kraus, "Negotiation and cooperation in multi-agent environments," *Artif. Intell.*, vol. 94, no. 1–2, pp. 79–97, 1997.
[34] S. S. Fatima, M. Wooldridge, and N. Jennings, "An analysis of feasible solutions for multi-issue negotiation involving non-linear utility functions," 2009.
[35] V. Konda and J. Tsitsiklis, "Actor-critic algorithms," *Adv. Neural Inf. Process. Syst.*, vol. 12, Nov. 1999.
[36] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
[37] OpenAI, "OpenAI API," 2024. [Online]. Available: https://platform.openai.com/docs/api-reference
[38] Groq, "Groqcloud API," 2024. [Online]. Available: https://groq.com/products/groqcloud/
[39] Anthropic, "Claude API," 2025. [Online]. Available: https://docs.anthropic.com/
[40] Google, "Gemini API documentation," 2024. [Online]. Available: https://ai.google.dev/api/docs
[41] HuggingFace, "Huggingface inference API," 2024. [Online]. Available: https://huggingface.co/docs/api-inference

Mohammad Wali Ur Rahman received the M.Sc. degree in electrical and computer engineering in 2023 from the University of Arizona, Tucson, AZ, USA, where he is currently working toward the Ph.D. degree.

He has been engaged as a Research Assistant with the Autonomic Computing Lab, a branch of the NSF Center for Autonomic Computing (NSF-CAC), Lubbock, TX, USA. His research interests include text mining and analysis, natural language processing, machine learning, neural networks, and artificial intelligence and cybersecurity.

Martin Manuel Lopez received the Ph.D. degree in electrical and computer engineering from the University of Arizona, Tucson, AZ, USA, in 2026.

He is currently a Technology and Management Consultant with Simulated Environments Inc (SEI), advising enterprise clients across life sciences, retail, defense, and utilities on AI enablement, enterprise architecture, and organizational transformation. His research interests include explainable AI through traceable multiagent agentic workflows for transparent and interpretable decision-making.

Lamia Tasnim Mim received the master's degree in computer science from the New Mexico State University, Las Cruces, NM, USA, in 2023.

She is currently a Graduate Assistant with New Mexico State University. She specializes in artificial intelligence, machine learning, and natural language processing, with a focus on developing robust and scalable deep learning models. She served as a Machine Learning Engineer at Avirtek, Inc., Tucson, AZ, USA, where she developed streaming data pipelines, deployed ML models, and enhanced data recovery accuracy through advanced techniques.

Carter Farthing received the B.S. degree in computer science from North Carolina State University, in 2001. He received the M.S. degree in information systems from University of Phoenix, in 2009. He is a Senior Engineer with the Technology Branch at the Department of Defense (DoD) Information Systems Agency (DISA) Joint Interoperability Test Command (JITC). He has 15 years of experience researching, integrating, configuring, and utilizing automation technologies for functional, performance, and standards conformance testing of DoD warfighter systems. Additionally, he has led command efforts in implementing DevSecOps (DSO) development solutions, integrating machine learning (ML) into test methodologies, and updating test capabilities to support Big Data requirements.

Julius Battle received the B.S. degree in electrical engineering from Virginia Commonwealth University, Richmond, VA, USA, in 2006 and the M.S. degree in information technology from the University of Maryland Global Campus, Adelphi, MD, USA, in 2014.

He previously served as the Lead Industrial Control Systems (ICS) Operational Technology Engineer for the Naval Facilities Engineering Command's (NAVFAC) cybersecurity startup division. He is currently a Data Scientist and AI Engineer with the Defense Information Systems Agency (DISA/JITC), where he develops AI pipelines and manages data science initiatives.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.

Kathryn Buckley received the B.S. degree in mathematics and information systems from Excelsior University, Albany, NY, USA, in 1985 and the M.B.A. degree in eCommerce and international business from Jones International University, Centennial, CO, USA, in 2004.

She is currently a Data Scientist and Operations Research Systems Analyst with over 15 years of experience at the Defense Information Systems Agency (DISA/JITC), applying statistical modeling, machine learning, and decision analytics to complex operational systems. She has led analytics and modeling initiatives across government and industry, supporting mission-critical systems, operational optimization, and data-driven decision making.

Salim Hariri (Senior Member, IEEE) received the M.Sc. degree in electrical engineering from Ohio State University, Columbus, OH, USA, in 1982, and the Ph.D. degree in computer engineering from the University of Southern California, California, CA, USA, in 1986.

He is currently a Professor with the Department of Electrical and Computer Engineering, University of Arizona, Tucson, AZ, USA, and the Director of the NSF Center for Cloud and Autonomic Computing (NSF-CAC). His research interests include autonomic computing, artificial intelligence, machine learning, cybersecurity, cyber resilience, and cloud security.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 07,2026 at 10:08:58 UTC from IEEE Xplore. Restrictions apply.